

Rerun Protocol

When Does Seasonal Decomposition Help?

Pipeline decisions, measurement and analysis plan, implementation,
testing, pilot and full runs

Prepared for D. Costa, the supervisor, and the partner university team

Version 2.3 — 8 October 2026 — v1.4 approved by the supervisor; later amendments in
Appendix C

What this document is. The single reference for the rerun of the study. It records every design decision and the reason for it, fixes the complete list of quantities to be measured and the analyses to be run on them *before* any data is produced, and lays out how the pipeline is implemented from the existing code, how it is tested, how the pilot is run and judged, and how the full runs are executed. After supervisor sign-off it is committed and tagged in the pipeline repository and frozen; later changes go only into the change log (Appendix C) with a date and a reason.

Bottom line. No existing forecast file is reused as a result. The existing code is reused after the changes in Section 6. One pilot, then one set of runs from one frozen manifest and one frozen configuration file, in strict mode, on four GPU machines. The analysis code is written and tested on pilot data before the full run starts.

Status of the decisions. Every decision in Section 2 was taken by D. Costa on 28 September 2026 in eight rounds of structured questions. The rejected alternatives and their reasons are recorded in Appendix B. The five technical specifications marked “Protocol” in the register (the eligibility rule derived from the length constraints, decomposition details, search spaces, the statistical cache and provenance fields) were not asked separately and are approved with the protocol at sign-off. The supervisor approves this protocol before implementation and the pilot’s go / no-go before the full run.

Contents

1	Purpose, scope and verdict on existing results	3
1.1	Why a rerun	3
1.2	Verdict: what is reused and what is not	3
1.3	Principles	3
2	Decision register	4
3	Data, eligibility, features and sampling	6
3.1	Data	6
3.2	Eligibility and the minimum-length constraint	7
3.3	Features	8
3.4	Sampling	8
3.5	Buckets	8
3.6	The manifest	8

4	Forecasting setup	9
4.1	Validation and decomposition	9
4.2	Models and classes	9
4.3	Tuning protocol	9
4.4	Training scopes and pairing	9
4.5	Failure semantics and determinism	10
4.6	Code, environment and data	11
5	Measurement and analysis plan	11
5.1	What is recorded	11
5.2	Derived quantities and aggregation rules	12
5.3	Practical significance	12
5.4	Confirmatory hypotheses	12
5.5	Secondary and robustness analyses	13
5.6	Tables and figures, mapped to the paper	14
6	Implementation plan	14
6.1	From scratch or from the current code	14
6.2	Module-by-module changes	14
6.3	Command-line interface	15
7	Testing and verification	16
7.1	Unit tests	16
7.2	Integration tests	17
7.3	Output gates	17
7.4	Traceability	18
8	Pilot	18
8.1	Scope	18
8.2	What the pilot must show	18
8.3	Exit criteria (go / no-go)	19
9	Full runs	19
9.1	Stages	19
9.2	Task grid	19
9.3	Compute estimate and allocation	19
9.4	Monitoring, failures and archival	20
10	Timeline	20
11	Risks	20
12	Consequences for the paper	21
A	Defects found in the review	21
B	Decision record: alternatives considered	22
C	Change log	24

1 Purpose, scope and verdict on existing results

1.1 Why a rerun

A review of the forecasting pipeline and of every run it produced (28 September 2026) found defects in the data the paper was built on and in the design of the experiment itself. The full list is in Appendix A; Table 1 summarises those that force a rerun.

Table 1: Defects that force a rerun. IDs are used throughout this document.

ID	Defect	Consequence
C1	The paper’s global baselines and all statistical STL results came from a non-strict run whose forecasts are a hard-coded blend of seasonal naive, drift and mean, jittered by $\pm 0.5\%$ per task.	Original Tables 3–8 do not describe any trained model.
C2	No global model is tuned. The “Auto” models are fixed configurations; tuning code writes empty placeholder files. The paper claims Optuna tuning over 100 trials.	Methods section is inaccurate; statistical models (self-selecting) are compared with untuned learners.
C3	Features are computed on the series minus only the last horizon, so they see the test targets of windows 0 and 1.	Bucket assignment, and every “which series” result, uses test data.
C4	The ML seasonal lag and the neural input size are set by the <i>shortest</i> series in each training pool; only 33% of quarterly cohort fits had the seasonal lag.	Both experiments are confounded by whether the model saw seasonal information.
M1	The two trained bodies of results use different series, and no trained global STL-AC run exists.	Specialisation cannot be measured on a paired basis.
F1	Features silently fall back to fixed values (0.0 or 1.0) on short series; 1.7–2.9% of the old sample.	Those series pile into the extreme buckets, e.g. 51 series in the Low evolving-seasonality bucket.

1.2 Verdict: what is reused and what is not

- **Not reused as results:** any forecast or metric file from the 23 May, 25 May or 28–30 May runs. The 23 May run is not model output (C1). The other two are genuine but carry C3 and C4, and fixing C4 changes the forecasts themselves; mixing old and new files would recreate the unpaired design (M1).
- **Reused as a check:** the 25 May strict run’s statistical forecasts. Statistical fits do not depend on the training pool, so the new code must reproduce them row for row on the same series (integration test I2).
- **Reused as code:** data loading, STL transform, rolling-origin split, RelNaive metric, the six feature definitions, checkpointing, scheduler and vast.ai scripts — after the changes in Section 6, in the new folder `pipeline/` of the project repository.
- **Reused as text:** introduction, related work, the 64 verified references, problem formulation and the structure of the methods and results sections.

1.3 Principles

Every decision in Section 2 follows from one of six principles.

- P1 One manifest, one configuration, one code version.** Every forecast in the study is traceable to a single frozen manifest, a single frozen configuration file and a single git commit.
- P2 Nothing from a test period is used before evaluation.** Features, sampling, buckets, tuning and scaling use only observations before the first test window.

- P3 A model’s configuration never depends on who else is in the pool.** Lags, input size and all hyperparameters come from the frozen configuration; series too short for them are excluded before sampling.
- P4 Every family receives comparable care.** Statistical models select their own specification; global models are tuned with a comparable, documented budget.
- P5 No silent failure path.** There is no fallback forecaster in the code. A failure is recorded as a failure and counted in the results.
- P6 The analysis refuses dirty inputs.** Automated gates check provenance, hashes, completeness and pairing before any number is computed.

2 Decision register

“Team” marks decisions taken by D. Costa in the structured questions of 28 September 2026; “Protocol” marks technical specifications approved with the protocol at sign-off. Rejected alternatives are in Appendix B; details in Sections 3–5.

ID	Topic	Decision	Rationale	By
<i>Scope and data</i>				
D1	Reuse	Full rerun. Legacy statistical forecasts used only as a regression check (I2).	Section 1.	Team
D2	Data	M3 and M4, monthly ($m = 12$, $h = 18$) and quarterly ($m = 4$, $h = 8$) only. One frozen copy, stored with SHA-256 checksums; every machine loads the same bytes. M3 from the original competition file <code>M3C.xls</code> with official ids, cross-checked against the Monash and Mcomp copies; M4 from the official competition files (amended in v1.3 and v1.4).	No new frequencies or loaders to validate; a changed mirror cannot change the data; the M3 data exactly as published.	Team
D3	Eligibility	A series is eligible if its history before the first test window has at least $L = 3m + h$ points: 54 monthly, 20 quarterly.	Every STL fit sees ≥ 2 seasons; tuning data holds $\geq 3m$ points; evolving seasonality gets ≥ 2 rolling windows. Fixes C4, M4.	Protocol
D4	Features	Definitions unchanged. Computed once, on the pre-window-0 history, for every eligible series. A series with any non-“ok” quality flag is ineligible.	Fixes C3 and F1; buckets fixed across windows; the follow-up gets features for the whole pool.	Team
D5	Feature guards	Option A: quarterly minimum lengths lowered to 20 (nonlinearity from 30, ARCH from 24); monthly guards (≤ 48) unchanged. Sensitivity analysis A7.	Keeps 600 per source and three windows (Section 3.2).	Team
D6	Sampling	Per-feature stratified quantile sampling, 30 strata, 600 series per source, one seed per feature, candidate pool = all eligible series.	The pool must exceed the quota for stratification to exist (M2).	Team
D7	Buckets	Terciles per feature $\times$ frequency within the feature’s sample. Degenerate if any populated bucket holds $< 20\%$: run with populated buckets, excluded from trend tests. Nonlinearity keeps its floored definition and is expected to be flagged.	Makes the collapse explicit; keeps the paper’s failure-mode contribution.	Team
D8	Validation	Rolling origin, $W = 3$ windows, step h ; every model refitted every window.	Standard protocol; all families treated alike.	Team
<i>Models and tuning</i>				

ID	Topic	Decision	Rationale	By
D9	Decomposition	Robust STL (statsmodels 0.14.6), fitted per window on training data only. Strategies Direct, STL-SN, STL-AC. STL failure is a failure; no fallback decomposition.	Fixes F2.	Protocol
D10	Models	The 12 models: ARIMA, SARIMA ($D = 1$), ETS; Ridge, LinearRegression, RandomForest, XGBoost; NLinear, NHITS, LSTM; TFT, PatchTST.	Design and text stay aligned.	Team
D11	Statistical	AutoARIMA with approximate stepwise search, AutoARIMA with $D = 1$, AutoETS (statsforecast 2.0.3); specification chosen per series by information criteria.	As in the valid May run; regression test I2 stays exact.	Team
D12	Global tuning	Library Auto classes with Optuna (TPE); 20 trials per study; the slowest family may drop to 15 only if the pilot projects an overrun, recorded before any real run. Objective: mean validation MASE.	Fixes C2; MASE is scale-free and native.	Team
D13	Tuning set	Fixed draw of 600 per source from the eligible pool; pre-window-0 data only; validation = one block, the last h points before the first cutoff. One study per model $\times$ frequency $\times$ target (raw, $T+R$, and T , S , R separately): 90 studies. Best configuration frozen to a hashed file.	Tuning never sees test data; each STL-AC component model is tuned for its own target.	Team
D14	Search spaces	Table 6; fixed a priori; identical for every pool; include scaling and training length.	P3; fixes C4, M5.	Protocol
D15	Training scopes	Cohort: one global model per feature sample $\times$ frequency, M3 and M4 pooled (1,200 series). Tercile: one per bucket (≈ 400). Same frozen configuration, same series, same windows; temporal hold-out on the same series.	Fully paired design; corrects the “held-out series” wording (M3).	Team
D25	ML inputs	Lags $1, \dots, m$; target transform tuned (none, local scaling, first difference, seasonal difference); no calendar or rolling features.	Same information as a seasonal autoregression; STL’s contribution measured cleanly.	Team
<i>Execution</i>				
D16	Failures	No fallback code. Statistical per-series failures recorded as failed rows. A global task is retried once; a second failure is fixed in a committed change and that task alone is rerun with the same manifest and configurations, logged.	P5; fixes C1, m2.	Team
D17	Seeds	One fixed seed per fit. Seed check: evolving seasonality, cohort scope, the 7 stochastic global models, 2 extra seeds (A15).	Quantifies seed sensitivity at modest cost.	Team
D18	Statistical cache	Statistical forecasts computed once per series $\times$ window $\times$ strategy $\times$ model, reused across feature samples.	Pool-independent; saves compute; identical values.	Protocol
D19	Provenance	Every row stores code commit, environment-lock hash, data checksum, manifest hash, configuration hash, backend and version, and — for STL-AC — all three component provenances.	Fixes m1, F3.	Protocol

ID	Topic	Decision	Rationale	By
D20	Infrastructure	Four GPU machines, sharded by family $\times$ frequency; outputs and logs synced continuously to a cloud bucket via rclone, with checksums.	Survives instance loss; merge refuses mismatched hashes.	Team
D22	Code & environment	The pipeline in a new top-level folder pipeline/ of the project repository, developed on branch rerun-v2 from an unmodified baseline commit (amended in v1.2); the May library versions pinned in a committed lockfile (Section 4.6).	One history for paper and code; every fix is a visible diff; regression test valid.	Team
<i>Analysis</i>				
D21	Metrics	Primary: RelNaive ⁽¹⁰⁾ . Secondary: unclipped RelNaive, MASE, sMAPE, POCID.	Continuity with the paper.	Team
D23	Inference	Series-level Wilcoxon signed-rank with Holm correction per predefined table; multiple comparison matrices; Friedman + Nemenyi critical-difference rankings for cohort (36 configurations) and tercile (27) scopes, overall and per frequency; bootstrap CIs; ordered-trend tests.	Section 5.	Team
D26	Practical threshold	An effect is a <i>gain</i> or <i>loss</i> only if Holm-significant and $ \text{median per-series } \Delta \geq 0.01$; significant but smaller effects are <i>negligible</i> .	With $\sim 10^6$ rows almost everything is significant.	Team
D27	Aggregation	Descriptives at instance level; inference at distinct-series level; families weighted equally when pooled.	Paper’s convention; no series or family counted twice.	Team
D28	Selector	No feature-based selector in this paper; stated as a limitation and left to the follow-up.	Scope.	Team
<i>Process</i>				
D24	Freeze	Protocol committed and tagged in the pipeline repository before stage R1; later changes only via the change log.	Prevents result-driven changes.	Team
D29	Release	Code, manifest and frozen result bundle released with the paper.	Every table and figure regenerable by reviewers.	Team
D30	Sign-off	The supervisor approves the protocol before implementation and the pilot’s go / no-go; the partner university is informed.	Accountability at both gates.	Team

3 Data, eligibility, features and sampling

3.1 Data

M3 and M4 monthly and quarterly series are downloaded once, validated, stored in the cloud bucket together with SHA-256 checksums, and loaded from that frozen copy on every machine (D2). The checksum is written into the manifest and every result row. Loading verifies it and refuses a mismatch (test U18).

Sources (D2, amended in v1.3 and v1.4). M3 is taken from the original competition file **M3C.xls** of the International Institute of Forecasters (M3 competition page), which holds the training and test values of every series, the official ids (**N0646**, ...) and start periods. The site refuses scripted downloads, so the file is supplied to the freeze, which refuses it unless its SHA-256 equals the value

pinned in the code and stores it with the raw files. Two further copies are compared with it value by value, and the freeze stops on any difference that is not documented: the Monash Time Series Forecasting Archive copy on Zenodo, which the earlier runs used, is identical at full precision (204,566 of 204,566 values); Mcomp 2.7 (CRAN; R. J. Hyndman) differs in one value, series N2786 at $t = 84$ (a test value), where it has 1200 and the original -1200 , without a note in its change log. The earlier runs read the Monash file through `datasetsforecast`, whose reader rounded 44,034 values to single precision (relative error $\leq 5.7 \times 10^{-8}$; defect m7); the freeze reads it at full precision. Series are named by their official M3 ids; the rows of `M3C.xls` are checked to be in official-id order, so the positional ids of the earlier runs (`M1`, ...) map one-to-one onto official ids, as Mcomp’s field `st` confirms. The start labels are kept as given, except that N1071’s “quarter 9 of 1984” is carried into 1986-Q1; the 29 monthly series without a start date keep their positions as labels. M4 is taken from the official competition files (`Mcompetitions/M4-methods`, `Dataset/Train` and `Dataset/Test`) and read by `read_m4_official`¹, which returns exactly what `M4.load` returns (checked against `M4.load` itself in the tests and, for M4 Quarterly, on the real files) and in addition refuses a training row with a gap, a test row without exactly h finite values, and ids that are duplicated or not matched one-to-one.

Validation and layout. Both sources are complete series: the competition’s test period is part of every series, and the rolling-origin windows of Section 4 are cut from them. The freeze stops unless every source has its published number of series (M3: 1,428 monthly, 756 quarterly; M4: 48,000 and 24,000) and its shortest series equals the published minimum training length plus h (M3: 48+18 and 16+8; M4: 42+18 and 16+8), which confirms that the series are complete. Each source is stored as one file with columns `unique_id`, `source_dataset`, `frequency`, position $t = 1, \dots, n$, y and the source’s time label (kept for traceability only). The manifest (`pipeline/data_manifest/m3m4_v1.json`) records the SHA-256 of every raw download and every frozen file, a content hash that does not depend on the file format, the results of the M3 cross-checks, and the code commit and package versions that produced the copy. The pipeline uses only the order of observations: the loader rebuilds one regular calendar from t , identical for M3 and M4, and has no fallback of any kind (defect F4, Appendix A).

3.2 Eligibility and the minimum-length constraint

With $W = 3$ windows of step h , the history available before the first test window is $n - 3h$. Table 3 lists every requirement on that history. The eligibility rule $L = 3m + h$ satisfies all modelling requirements. The only conflicts were the quarterly guards of nonlinearity and ARCH, resolved by D5.

Table 3: Minimum pre-window-0 length required by each component.

Requirement	Monthly	Quarterly	Source
STL with ≥ 2 seasons in the tuning data (pre-window-0 minus h)	42	16	D3
ML lags $\leq m$ plus seasonal differencing at the tuning origin	42	16	D25
Evolving seasonality: ≥ 2 rolling windows (window + step)	42	18	feature code
Non-normality test	20	20	feature code
Structural breaks	48	20	feature code
ARCH test (quarterly: original 24, lowered by D5)	48	20	feature code
Nonlinearity (quarterly: original 30, lowered by D5)	48	20	feature code
Eligibility rule $L = 3m + h$	54	20	D3

Table 4 gives the eligible pool under the adopted option and the alternatives that were considered, estimated from the 25 May run’s cutoffs; exact counts are produced by stage R1.

¹`src/data/freeze_data.py`; `M4.load` of `datasetsforecast` melts the whole wide table and was stopped by the out-of-memory killer on a machine with 7.6 GiB.

Table 4: Estimated eligible series. A (adopted): three windows, quarterly guards lowered to 20. B: two quarterly windows, guards of 24. C: three windows, original guards (30). D: three windows, guards of 24. Monthly figures are the same under every option, because the monthly rule (54) already exceeds every monthly guard (48).

Source	Pool	A (adopted)	B	C	D
M3 Monthly	1,428	1,040	1,040	1,040	1,040
M3 Quarterly	756	634	666	116	433
M4 Monthly	48,000	35,000	35,000	35,000	35,000
M4 Quarterly	24,000	22,500	23,400	20,100	20,700

Under option A, nonlinearity and ARCH are computed on 20–29 points for the shortest quarterly series and are noisier there; analysis A7 repeats the feature results on the series that meet the original guards. M3 Quarterly is close to a census (600 of about 634 eligible series) and M3 Monthly supplies 600 of about 1,040, so per-feature stratification is substantive only for M4. The paper states this.

3.3 Features

The six feature definitions of the paper are unchanged, with three changes to how they are computed (D4).

1. Each feature is computed once, on $y_{1:n-3h}$, the history before the first test window. Features and buckets are the same for all three windows.
2. Any series with a non-“ok” quality flag on any feature is removed from the eligible pool, so no fallback value ever enters a sample (fixes F1).
3. Features are computed for *every* eligible series, about 59,000, so the follow-up meta-learning paper has all six features for every eligible series.

3.4 Sampling

For each feature k , frequency and source independently:

1. Rank the eligible series by ϕ_k (ties broken by series id).
2. Cut the ranking into 30 equal-count strata.
3. Draw $600/30 = 20$ series per stratum without replacement, seed = $s_0 + k$.
4. Record the stratum, rank and quantile of every selected series in the manifest.

Each feature sample holds 600 series per source: 1,200 per frequency and 2,400 in total. Across the six features, about 8,500 distinct series are expected. M4 samples are nearly disjoint; M3 samples overlap heavily. Sampling quality is checked by gate G11.

3.5 Buckets

Within each feature sample and frequency, the 1,200 series are split at the empirical 1/3 and 2/3 quantiles into Low, Medium and High, about 400 each. If any populated bucket holds less than 20% of the sample, the feature is flagged degenerate for that frequency (gate G12). It is then run with its populated buckets only and excluded from the tercile-trend tests (H4). The nonlinearity feature keeps its floored definition (D7) and is expected to trigger this guard, as documented in the paper’s Section 4.5.

3.6 The manifest

Stage R1 writes one manifest bundle, which is hashed and frozen. It contains:

- the data checksum;

- the eligibility table for the full pool (length, flags, eligible yes/no);
- feature values for all eligible series;
- the six feature samples with strata and seeds;
- bucket assignments;
- the tuning set;
- the cutoffs for all windows.

Nothing downstream re-samples, re-buckets or recomputes cutoffs.

4 Forecasting setup

4.1 Validation and decomposition

For series i and window $w \in \{0, 1, 2\}$, the training set ends at $n_i - (3 - w)h$ and the test set is the next h points. Every model — statistical and global — is refitted for every window (D8). STL is fitted on each window’s training data only. Its three strategies are:

- **Direct:** the model forecasts the raw series.
- **STL-SN:** a model forecasts $T + R$, and the last seasonal cycle of S is repeated and added back.
- **STL-AC:** one separately trained model per component, and the forecasts are summed.

4.2 Models and classes

The Auto classes are used only in the tuning phase. The evaluation phase instantiates the plain classes from the frozen configuration file, so every evaluation forecast is fully determined by that file. The Auto class names and signatures are verified in the pinned environment on day 1 (test I5), because the libraries are not installed on the development machine.

4.3 Tuning protocol

- **Data:** the tuning set (D13), truncated at each series’ first cutoff. The last h points form a single validation block; the rest is training data. No tuning step sees a test period (gate G7).
- **Studies:** one per global model $\times$ frequency $\times$ target, i.e. $9 \times 2 \times 5 = 90$. The five targets are the raw series (Direct), $T + R$ (STL-SN), and T , S , R separately (STL-AC). 50 studies run on GPU (neural and transformer) and 40 on CPU (ML).
- **Search:** Optuna TPE sampler, 20 trials, fixed sampler seed. If the pilot projects an overrun, the slowest family may drop to 15 trials; the change is recorded before any real run (D12). Studies are stored in SQLite and archived.
- **Objective:** mean validation MASE with seasonality m . NeuralForecast is configured with `valid_loss=MASE(seasonality=m)`; AutoMLForecast with an equivalent loss function.
- **Early stopping:** used during tuning only. In evaluation, the number of training steps is part of the frozen configuration, since no validation data exists after a test cutoff.
- **Freezing:** the best trial of each study is written to `configs_frozen.json`, whose SHA-256 hash is stored in every result row. Tercile-scope models use exactly the same configuration as cohort-scope models (D15).

4.4 Training scopes and pairing

For each feature sample, frequency, strategy, model and window, the cohort model is trained on all 1,200 series of that frequency — M3 and M4 pooled — and the three tercile models on about 400 each. All four use the same frozen configuration and are evaluated on the same series and windows.

Table 5: Model classes used in each phase.

Family	Model	Tuning phase	Evaluation phase
Statistical	ARIMA	— (IC selection)	<code>AutoARIMA</code> (season_ length=m), ap- prox- i- mate
Statistical	SARIMA	—	<code>AutoARIMA</code> (season_ length=m, D=1), ap- prox- i- mate
Statistical	ETS	—	<code>AutoETS</code> (season_ length=m)
ML	Ridge / LinReg / RF / XGBoost	<code>AutoMLForecast</code> + model spaces	<code>MLForecast</code> / NL / NL with frozen con- fig
Neural	NLinear, NHITS, LSTM	<code>AutoNLinear</code> , <code>AutoNHITS</code> , <code>AutoLSTM</code>	plainL / NL / NL classes, frozen con- fig
Transformer	TFT, PatchTST	<code>AutoTFT</code> , <code>AutoPatchTST</code>	plainNL / NL classes, frozen con- fig

Every tercile forecast therefore has exactly one cohort twin, and gate G6 enforces this. Δ_{FT} isolates the effect of the narrower training pool.

Statistical models appear only in the cohort scope. Their forecasts do not depend on the pool, so they are computed once per series, window, strategy and model and reused across feature samples (D18).

4.5 Failure semantics and determinism

- **No fallback forecaster.** `baselines.py`, the component stub and the per-task jitter are deleted. The result object rejects any source other than `trained_*` (test U9).
- **Statistical models:** a per-series exception produces a *failed row* with the exception class and message, and no forecast. The task continues.
- **Global models:** an exception fails the whole task, which is retried once automatically. If it fails again, the cause is fixed in a committed code change and only that task is rerun, with the same manifest and configurations; the change log records it (D16).
- **STL-AC:** if any of the three component models fails, the row fails. All three component provenances are stored.
- **Seeds:** one fixed seed per fit, with deterministic backend flags where available. Forecast hashes are stored so that any rerun can be compared exactly. Seed sensitivity is measured by the seed

Table 6: Search spaces (D14). They are identical for every pool. The pilot may only *narrow* a space for feasibility, and any such change goes into the change log.

Scope	Space
All neural and transformer	<code>input_size</code> : monthly $\{18, 36\}$, quarterly $\{8, 16\}$; <code>scaler_type</code> $\in \{\text{identity, standard, robust}\}$; learning rate log-uniform $[10^{-4}, 10^{-2}]$; <code>max_steps</code> $\in \{500, 1000, 2000\}$; batch size $\in \{32, 64\}$; <code>start_padding_enabled=True</code> (fixed).
NLinear	common parameters only
NHITS	<code>n_pool_kernel_size</code> $\in \{[2, 2, 1], [1, 1, 1]\}$; <code>n_freq_downsample</code> $\in \{[4, 2, 1], [1, 1, 1]\}$; MLP units $\in \{2 \times 64, 2 \times 256\}$
LSTM	encoder hidden $\in \{32, 64, 128\}$; encoder layers $\in \{1, 2\}$; decoder hidden $\in \{32, 64\}$
TFT	hidden size $\in \{32, 64, 128\}$; heads $\in \{2, 4\}$; dropout $\in \{0, 0.1, 0.2\}$
PatchTST	hidden size $\in \{32, 64, 128\}$; heads $\in \{4, 8\}$; encoder layers $\in \{1, 2, 3\}$; patch length $\in \{4, 8\}$, capped at <code>input_size</code> ; stride = patch length / 2
All ML (D25)	lags $1, \dots, m$ (fixed); target transform $\in \{\text{none, local standard scaling, first difference, seasonal difference}\}$; no calendar or rolling features
Ridge	α log-uniform $[10^{-3}, 10^2]$
LinearRegression	target transform only
RandomForest	trees $\in \{100, 300\}$; max depth $\in \{\text{none}, 10, 20\}$; min samples per leaf $\in \{1, 5, 20\}$; max features $\in \{1.0, 0.5, \text{sqrt}\}$
XGBoost	trees $[100, 600]$; learning rate log-uniform $[0.01, 0.3]$; max depth $[3, 9]$; subsample $[0.6, 1]$; column sample $[0.6, 1]$

check (D17, A15).

4.6 Code, environment and data

The pipeline lives in the new top-level folder `pipeline/` of the project repository (github.com/DiogoCosta18/TimeSeries) and is developed on branch `rerun-v2`, starting from an unmodified baseline commit; the paper and the code therefore share one history, and the results record the commit that produced them (D22). The environment is pinned to the versions of the valid May runs: Python 3.11.10, torch 2.5.1+cu121, mlforecast 1.0.31, neuralforecast 3.1.8, statsforecast 2.0.3, optuna 4.8.0, statsmodels 0.14.6, scikit-learn 1.8.0, numpy 2.4.6, pandas 2.3.3, scipy 1.15.3, ruptures 1.1.10 and datasetsforecast 1.0.1, plus `rdata 0.11.2` for the Mcomp cross-check (added in v1.3); `M3C.xls` is read with `xldr 2.0.2`. A lockfile is committed and its hash is stored in every row. The same image is used on every machine, and all machines load the frozen data copy (D2).

5 Measurement and analysis plan

This section is frozen before the full run (D24). The analysis code implements it and is tested on pilot data first (I3).

5.1 What is recorded

- **Per forecast row** (task $\times$ series $\times$ window): forecast and seasonal-naive vectors; RelNaive⁽¹⁰⁾ and unclipped RelNaive; MASE, sMAPE, MAE, POCID; the keys (feature sample, frequency, strategy, family, model, scope, bucket, series, source, window, cutoff, seed); training and inference time; status (trained / failed) with the failure reason; and the full provenance of D19.
- **Per series**: length, eligibility, all six features with flags, sample membership, stratum, bucket per feature.

- **Per tuning study:** every trial’s parameters, validation MASE and duration; the best configuration; the SQLite study file.
- **Per run:** task-level logs, heartbeat, resource usage, gate reports.

5.2 Derived quantities and aggregation rules

- **Instance:** (feature sample, series, window, model, strategy, scope, seed).
- $\Delta_{\text{STL}} = \text{RelNaive}_{\text{Direct}}^{(10)} - \text{RelNaive}_{\text{STL}}^{(10)}$, paired on everything except the strategy. $\Delta_{\text{FT}} = \text{RelNaive}_{\text{cohort}}^{(10)} - \text{RelNaive}_{\text{tercile}}^{(10)}$, paired on everything except the scope. Positive values mean the treatment helps.
- **Descriptives** (mean of capped values, median, win rate = share of instances with $\Delta > 0$, tie share) are computed at the *instance* level. This keeps the convention of the current paper (D27).
- **Inference** is at the *distinct-series* level. A series’ value is its mean over the instances in the reporting group — windows, models and, where a series appears in several feature samples, those samples. No series is counted twice.
- **Family pooling:** when families are pooled, each family is weighted equally (mean of family means), so the four-model ML family does not outweigh the two transformers (D27).
- **Seed check:** only the main seed enters H1–H7; the extra seeds are used by A15 only.
- **Failures:** pairwise deletion. Every table reports the number of excluded instances.

5.3 Practical significance

With about 2.8×10^6 forecast rows, almost every difference is statistically significant. Every confirmatory contrast is therefore labelled by the rule of D26:

- **Gain / loss:** Holm-significant and $|\text{median per-series } \Delta| \geq 0.01$, i.e. at least 1% of the seasonal-naive error, in the direction of the sign.
- **Negligible:** Holm-significant but $|\text{median per-series } \Delta| < 0.01$.
- **No evidence:** not significant after Holm correction.

For H4 the contrast is the difference in median per-series Δ_{STL} between the High and Low terciles; for H6 it is the median of the per-series paired difference. The paper uses these three labels verbatim; “gain” is never used for a negligible effect.

5.4 Confirmatory hypotheses

All tests are two-sided at $\alpha = 0.05$, with Holm correction within the stated family of tests. Effect sizes are reported with every test: the median per-series Δ with a 95% bootstrap confidence interval (2,000 resamples of series) and the matched-pairs rank-biserial correlation.

ID	Question	Test and unit	Correction family
H1	Does STL change accuracy, per family and strategy (cohort scope)?	Wilcoxon signed-rank on per-series Δ_{STL}	Holm: 4 families $\times$ 2 strategies = 8
H2	Does the STL effect differ between families?	Paired Wilcoxon on per-series Δ_{STL} between families (same series)	Holm: 6 pairs $\times$ 2 strategies = 12
H3	Which strategy is best within each family?	Paired Wilcoxon on $\text{RelNaive}^{(10)}$, Direct vs STL-SN vs STL-AC; shown as multiple comparison matrices	Holm: 3 pairs $\times$ 4 families = 12

ID	Question	Test and unit	Correction family
H4	Does the STL effect vary with each feature?	(a) Jonckheere–Terpstra trend test of per-series Δ_{STL} across Low < Medium < High; (b) Spearman ρ between feature value and per-series Δ_{STL} , bootstrap CI. Family-balanced pooling; per family as secondary.	Holm: 6 features $\times$ 2 strategies = 12 per test type
H5	Does bucket specialisation change accuracy, per family and strategy?	Wilcoxon on per-series Δ_{FT}	Holm: 3 families $\times$ 3 strategies = 9
H6	Do linear and non-linear models respond differently to specialisation?	Paired Wilcoxon on per-series [mean Δ_{FT} of non-linear models – mean Δ_{FT} of linear models]	Holm: 3 strategies = 3
H7	Do configurations differ in overall rank?	Friedman test on per-series mean RelNaive ⁽¹⁰⁾ with distinct series as blocks, then Nemenyi post-hoc and critical-difference diagrams: cohort scope (36 configurations = 12 models $\times$ 3 strategies) and tercile scope (27 = 9 $\times$ 3), each overall and per frequency — six diagrams.	Nemenyi’s own family-wise control within each diagram

For H7, the critical difference shrinks with the number of blocks, so with thousands of series most pairs will be separated in rank. A rank difference is read as practically meaningful only where the corresponding paired contrast also meets the D26 threshold.

5.5 Secondary and robustness analyses

- A1** Model standing: RelNaive⁽¹⁰⁾ by family $\times$ model $\times$ strategy $\times$ scope; top configurations; distributions.
- A2** Frequency split: H1, H4 and H5 repeated for monthly and quarterly separately.
- A3** Source split: M3 vs M4.
- A4** Window consistency: sign and size of Δ_{STL} and Δ_{FT} per window.
- A5** Metric sensitivity: H1 and H5 on MASE, sMAPE and unclipped median RelNaive.
- A6** Cap sensitivity: means at caps 5, 10 and 20, and 5% trimmed means.
- A7** Guard sensitivity (D5): H4 for nonlinearity and ARCH restricted to quarterly series with ≥ 30 and ≥ 24 pre-window-0 points respectively.
- A8** Failure accounting by family, model, strategy and scope.
- A9** Compute cost: training and inference time by family and strategy.
- A10** Tuning report: frozen configurations, trial curves, best vs first trial.
- A11** Sampling quality: Kolmogorov–Smirnov distance between sample and eligible pool per feature and source.
- A12** Bucket support and degenerate-feature report.
- A13** Per-model tables for Δ_{STL} and Δ_{FT} .
- A14** Correlations between the six features on the eligible pool.
- A15** Seed sensitivity (D17): for evolving seasonality in the cohort scope, the spread of per-series RelNaive⁽¹⁰⁾ across three seeds for the seven stochastic global models, and whether any H1 or H3 label (gain / loss / negligible / no evidence) changes when another seed is used.

Table 8: Outputs produced by the analysis package. All are generated by committed scripts from the frozen result bundle.

Paper section	Output	Analysis
§2 Data	Pool / eligible / sampled counts; sampling illustration; KS table	D3, A11
§2 Models	Model table; search spaces; frozen configurations (appendix)	D10–D14, A10
§2 Features	Feature distributions; bucket sizes with support check	A12, A14
§3.1	Top configurations per scope; RelNaive ⁽¹⁰⁾ distributions by family $\times$ strategy; cohort-scope critical-difference diagrams	A1, H7
§3.2	Δ_{STL} by family $\times$ strategy with CIs and D26 labels; bar chart; MCM per family	H1–H3
§3.3	Δ_{STL} by feature $\times$ tercile (per strategy); trend and Spearman tables; family heatmap	H4, A7
§4	Δ_{FT} overall, by bucket, family, model, strategy; linear vs non-linear; significance counts; tercile-scope critical-difference diagrams	H5–H7, A13
Appendix	Frequency, source, window, metric, cap and seed robustness; failures; compute cost	A2–A9, A15

5.6 Tables and figures, mapped to the paper

6 Implementation plan

6.1 From scratch or from the current code

Table 9: Rewrite versus refactor.

	From scratch	Refactor the current code (adopted)
For	No hidden legacy path survives; cleaner architecture.	STL, split, metric, features, checkpointing and vast.ai scripts are already correct and tested; feature code <i>is</i> the paper’s definition.
Against	Four days is not enough to rebuild and re-test loading, features, checkpointing and orchestration; new bugs; features must be re-validated against the old definitions.	Every unsafe path must be found. Mitigated by deleting rather than disabling them, and by static test U9.

The current code is copied unchanged into the folder `pipeline/` of the project repository, committed as a baseline, and refactored on a branch `rerun-v2`. Unsafe paths are deleted, not switched off; the non-strict mode ceases to exist.

6.2 Module-by-module changes

File	Change	Fixes
<code>src/data/freeze_data.py</code> (new)	One-off download of M3/M4 into the cloud bucket with SHA-256 checksums; validation; M3 from <code>M3C.xls</code> with the Monash and Mcomp cross-checks; memory-safe M4 reader.	D2, m7
<code>src/data/frozen.py</code> (new)	Canonical layout, content hash, verified loading.	D2
<code>src/data/load_m_datasets.py</code>	Load only from the frozen copy; verify checksums; refuse a mismatch; one regular calendar from the position.	D2, F4, m6
<code>src/data/eligibility.py</code> (new)	Eligibility table: length rule L , feature flags; refuses ineligible series downstream.	C4, M4, F1

File	Change	Fixes
src/data/validation.py	<code>make_cutoffs</code> asserts eligibility instead of skipping short windows.	M4
src/features/compute_all.py	Compute on $y_{1:n-3h}$ (line 66); a failed feature marks the series ineligible instead of writing default values.	C3, F1
src/features/nonlinear.py, arch.py	Quarterly guards lowered to 20 (D5).	D5
src/features/mstl_features.py	<code>decompose_series</code> : remove constant-trend and moving-average fallbacks; raise instead.	F2
src/data/sample_series.py	600 per source from the eligible pool; per-feature seed; strata recorded; 20% minimum-support guard in bucket assignment.	M2, D7
src/models/baselines.py, component_wrappers.py	Deleted.	C1
src/models/*_auto.py	Split into <code>tune()</code> (Auto classes, Optuna, MASE) and <code>fit_predict()</code> (plain classes with a frozen config). Remove <code>_safe_lags</code> , <code>_safe_input_size</code> and all fallback branches.	C2, C4
src/models/statistical.py	Approximate AutoARIMA kept; seasonal-naive fallback removed; per-series failures raise a recorded failure.	C1, m2
src/models/forecast_result.py	Accept only <code>trained_*</code> sources and failed status.	C1
src/training/train_family.py	Delete <code>_identity_scale</code> , <code>_apply_identity_adjustment</code> and <code>uid_not_in_global_result</code> ; add component provenance, pairing keys, seed key and the statistical cache.	C1, m1, D17, D18
src/training/tune.py	Real tuning: study creation, SQLite storage, 20-trial budget, frozen-config file with hash.	C2
src/training/strict_mode.py	Removed; strictness is unconditional.	C1
src/training/task_graph.py	Scopes {cohort, low, medium, high} plus seed-check tasks; delete <code>finetune_by_feature_bucket</code> and per-series modes.	M1, D17
src/training/run_experiment.py	Split into stages <code>prepare</code> , <code>tune</code> , <code>evaluate</code> , <code>merge</code> , <code>gates</code> , <code>analyse</code> ; hashes written at each stage; logs synced.	M1, m5
src/training/scheduler.py	Shard selection by family $\times$ frequency; idempotent per-task outputs; rclone sync to the bucket.	D20
src/validation/gates.py (new)	Gates G1–G13 (Section 7.3).	m3
src/analysis/ (new)	Replaces <code>rebuild_paper_data.py</code> and the scratch scripts: loading with gates, deltas, Wilcoxon + Holm, MCM, Friedman + Nemenyi, D26 labels, tables, figures.	m3, m4
configs/rerun_v2.yaml (new)	Encodes every decision in Section 2; the old budget modes that permit non-strict runs are removed.	—

6.3 Command-line interface

```
python -m src.cli freeze-data --out <bucket>/data          # once, before R1
python -m src.cli prepare  --config configs/rerun_v2.yaml --run runs/v2
```

```
python -m src.cli tune      --run runs/v2 --shard <family>-<frequency>
python -m src.cli freeze    --run runs/v2          # writes configs_frozen.json + hash
python -m src.cli evaluate  --run runs/v2 --shard <family>-<frequency>
python -m src.cli merge     --run runs/v2          # refuses mismatched hashes
python -m src.cli gates     --run runs/v2          # G1-G13, blocks analysis on failure
python -m src.cli analyse   --run runs/v2 --out <paper-repo>/results_v2
```

7 Testing and verification

Three layers: unit tests on code, integration tests on end-to-end runs, and gates on every produced output. A failing gate blocks the next stage.

Tests inherited from the baseline that encode behaviour this protocol replaces are rewritten or deleted in the commit that replaces that behaviour, with the reason in the commit message; every other test passes at every commit. At the baseline four inherited tests fail: two expect the old task graph (972 tasks; a per-bucket mode for statistical models) and two expect the old strict leaderboard to reject zero or missing training times. They are resolved with the task graph and provenance (D19) work.

7.1 Unit tests

ID	Asserts	Guards
U1	Perturbing any value after the first cutoff leaves all six features unchanged.	C3
U2	Series with fewer than L pre-window-0 points are ineligible; <code>make_cutoffs</code> raises on them.	C4, M4
U3	On synthetic series of length exactly L , all six features return “ok”.	D5, F1
U4	No sampled series carries a non-“ok” feature flag.	F1
U5	STL: decomposition raises on too-short input; $T + S + R = y$ to 10^{-9} ; components unchanged when test data is perturbed.	F2, C3
U6	RelNaive, the cap and the seasonal naive match hand-computed examples.	metric
U7	Sampling is deterministic given the seed, draws 600 per source, fills strata quotas, and uses different seeds per feature.	M2
U8	Tercile sizes differ by at most one; the 20% support guard triggers on a degenerate synthetic feature.	D7
U9	Static check: no module can produce a non-trained forecast source; <code>baselines.py</code> is absent; the result object rejects other sources.	C1
U10	The same task run twice gives identical forecast hashes (CPU backends; neural with a fixed seed on CPU).	C1 jitter
U11	Instantiated models carry exactly the frozen configuration, whatever the composition of the pool.	C4, C2
U12	Tuning data ends before each series’ first cutoff; validation is the last h points before it.	C2, C3
U13	STL-AC rows store three component provenances; any component failure fails the row.	m1
U14	Every tercile row has exactly one cohort twin; a missing twin fails the merge.	M1
U15	Merge refuses shards with a mismatched manifest, configuration, code, environment or data hash.	M1, m3
U16	Statistical forecasts are identical across feature samples for the same series and window.	D18
U17	A statistical per-series exception yields a failed row and the task continues; a global exception fails the task with no forecast rows.	D16

ID	Asserts	Guards
U18	Loading refuses data whose checksum differs from the frozen copy, a missing file, an unlisted source and a configuration that does not match the manifest; the loader contains no exception handler, downloader or random generator.	D2, F4
U19	Wilcoxon, Holm, Friedman, Nemenyi, Jonckheere–Terpstra and the D26 labelling match reference implementations and hand-computed toy cases.	D23, D26

Existing tests. Kept: `test_rel_naive`, `test_component_recomposition`, `test_stl_no_leakage` (extended to U5), `test_checkpoint_resume`, `test_checkpoint_schema`, `test_feature_parallel_cache`, `test_timing_provenance`. Updated: `test_representative_sampling`, `test_parallel_scheduler`, `test_global_model_adapters`, `test_feature_scaling_bins`, `test_benchmark_correctness`. Rewritten: `test_feature_fallbacks` (fallback now means ineligible) and `test_strict_provenance` (no fallback path exists). Replaced: `test_honest_optuna_status`, by U11 and U12.

7.2 Integration tests

- I1 Smoke:** synthetic data, all families, one trial per study, CPU. The full chain freeze-data → prepare → tune → freeze → evaluate → merge → gates → analyse passes.
- I2 Regression:** on the eligible subset of the 25 May strict run’s manifest, the new code reproduces its AutoARIMA and AutoETS Direct metrics row for row ($|\text{diff}| < 10^{-6}$).
- I3 Analysis dry run:** the analysis package produces every table and figure of Table 8 from pilot outputs.
- I4 Resume:** a task killed mid-run and resumed produces identical outputs.
- I5 API check (day 1):** every class and argument in Table 5 and Section 4.3 exists in the pinned environment.

7.3 Output gates

ID	Passes if	Stage
G1	Every forecast row is <code>trained_*</code> ; failures appear only in the failure table.	merge
G2	All shards carry the frozen manifest hash.	merge
G3	All shards carry the frozen configuration hash.	merge
G4	All shards carry the same code commit (clean tree) and environment-lock hash.	merge
G5	Task grid complete: expected – completed = listed failures; global tasks have no unresolved failures; statistical per-series failures $\leq 0.5\%$, else investigated.	merge
G6	Pairing complete for Δ_{STL} (every STL row has a Direct twin) and Δ_{FT} (every tercile row has a cohort twin).	merge
G7	Logged feature and tuning end indices all precede the first cutoff.	prepare, tune
G8	No manifest series has fewer than L pre-window-0 points.	prepare
G9	No NaN or infinite metric; share of capped RelNaive $\leq 2\%$.	merge
G10	STL-AC / STL-SN median training-time ratio in $[2, 4]$ per global family.	merge
G11	Kolmogorov–Smirnov $p \geq 0.01$ between sample and eligible pool for each feature and M4 source; M3 reported.	prepare
G12	Every populated bucket $\geq 20\%$ of its sample, or the feature is flagged degenerate.	prepare

ID	Passes if	Stage
G13	Every shard carries the frozen data checksum.	prepare, merge

7.4 Traceability

Table 13: Every defect is closed by a decision, a test and a gate.

Defect	Decision	Unit tests	Integration	Gates
C1 fallback stub, jitter	D16	U9, U10, U17	I1	G1
C2 no tuning	D12–D14	U11, U12	I5	G3
C3 feature leakage	D4	U1, U5	—	G7
C4 pool-dependent configs	D3, D14, D25	U2, U11	—	G8
M1 unpaired arms	D15, D20	U14, U15	I1	G2, G6
M2 no stratification	D6	U7	—	G11
M3 “held-out series”	D15	—	—	paper text
M4 short histories	D3	U2	—	G8
M5 thin neural training	D14	U11	—	—
F1 feature fallback values	D4, D5	U3, U4	—	G8
F2 STL fallback	D9	U5	—	—
F3 code not versioned	D22	—	—	G4
F4 silent synthetic data	D2	U18	—	G13
m1 component provenance	D19	U13	—	—
m3 unchecked merge	D20	U15	I3	G2–G6, G13
m6 M4 time stamps	D2	U18	—	—
m7 M3 values	D2	freeze cross-check	—	G13

8 Pilot

8.1 Scope

- Two features: evolving seasonality (the strongest earlier signal) and nonlinearity (it exercises the degenerate-bucket guard).
- Both frequencies; 100 series per source per feature (400 per feature).
- All 12 models, 3 strategies, 4 scopes, 3 windows, main seed only.
- Tuning: all 90 studies with 3 trials each, on a tuning set of 100 per source.
- One GPU machine, the same image and frozen data copy as the full run.

8.2 What the pilot must show

1. All unit and integration tests pass, including I2 (regression) and I3 (analysis dry run).
2. All gates G1–G13 pass on the pilot outputs, with G12 flagging nonlinearity as expected.
3. Measured time per task and per trial, by family, frequency and strategy. These replace the estimates of Section 9 and set the shard allocation.
4. Peak GPU memory per model at the largest configuration in each search space.
5. Failure rate by family, with every failure explained.

8.3 Exit criteria (go / no-go)

Go if all tests and gates pass, there is no unexplained failure, and the projected wall-clock time of the full run on four machines is at most three days. The supervisor takes the go / no-go decision (D30). The only permitted adjustments before going are:

- narrowing a search space for memory or time;
- reducing trials from 20 to 15 for the slowest family (D12);
- re-balancing shards.

Each adjustment is recorded in the change log. Results from the pilot are never reported.

9 Full runs

9.1 Stages

Stage	Name	Content	Output (frozen)
R0	Freeze code	Tag the commit (with the signed-off protocol); build and hash the environment image; data copy already frozen.	commit tag, image hash, data checksum
R1	Prepare (CPU)	Eligibility on the full pool; features on pre-window-0 data for all eligible series (about 59,000); six samples; buckets; tuning set; cut-offs. Gates G7, G8, G11, G12, G13.	manifest bundle + hash
R2	Tune	90 studies, sharded (50 GPU, 40 CPU). Gate G7.	study files
R2b	Freeze configs	Best trial per study written and hashed.	<code>configs_frozen.json</code> + hash
R3	Evaluate	Up to 1,488 tasks, sharded; statistical cache shared; cohort shards first, then tercile, then seed check. Each shard is validated on completion.	shard outputs + logs
R4	Merge + gates	Merge; G1–G6, G9, G10, G13. Retry or fix any failed global task (D16).	result bundle + hash
R5	Analyse	Analysis package on the frozen bundle.	tables, figures, reports

9.2 Task grid

- **Cohort scope:** 6 features $\times$ 2 frequencies $\times$ 3 strategies $\times$ 12 models = 432 tasks. Of these, 108 are statistical and are served by the cache.
- **Tercile scope:** 6 $\times$ 2 $\times$ 3 $\times$ 9 $\times$ 3 buckets = 972 tasks. This falls to 918 if nonlinearity has an empty Medium bucket in both frequencies.
- **Seed check:** 1 feature $\times$ 2 $\times$ 3 $\times$ 7 stochastic models $\times$ 2 extra seeds = 84 tasks. Ridge and LinearRegression are deterministic and excluded.
- **Total:** up to 1,488 tasks, about 2.8 million forecast rows.

9.3 Compute estimate and allocation

The basis is the previous runs on one machine with 64 cores, one 24 GB GPU and fixed small configurations: 288 cohort-style tasks in about 12 hours, and 972 tercile tasks in about 50 hours. At the new task count this is about 70 machine-hours. Tuned configurations may cost 1.5–3 times more, giving roughly 105–210 machine-hours of evaluation. Tuning adds 50 GPU studies $\times$ 20 trials at an estimated 0.5–3 minutes each, i.e. 8–50 GPU-hours, plus CPU studies in parallel.

Initial allocation of the four machines (D20), re-balanced from the pilot’s measured times:

- machine A: neural monthly;

- machine B: neural and transformer quarterly;
- machine C: transformer monthly;
- machine D: ML and statistical, both frequencies (CPU-bound).

Expected wall-clock time: tuning 2–13 hours, evaluation 26–53 hours.

9.4 Monitoring, failures and archival

- Heartbeat and status every minute; checkpoints every five minutes.
- Outputs and logs synced to the cloud bucket at every checkpoint via rclone, with checksums, so no machine holds the only copy (D20).
- A failed global task is retried once automatically. A second failure is investigated; the fix is committed, and the task alone is rerun with the same manifest and configuration (D16).
- **Priority:** cohort-scope shards (Experiment 1) run first, tercile-scope shards second and the seed check last, so an overrun cannot leave the main result incomplete.
- **Archival:** SHA-256 checksums of every file. The final bundle is read-only; every number in the paper is generated from it by committed scripts, and code, manifest and bundle are released with the paper (D29).

10 Timeline

Day 1 is the day of supervisor sign-off; the dates assume Tuesday 29 September 2026.

Table 15: Two-week plan.

Day	Date	Work
1	Tue 29 Sep	Supervisor sign-off. Folder <code>pipeline/</code> ; baseline commit; branch <code>rerun-v2</code> ; environment lockfile; I5 API check; frozen data copy with checksums (U18).
2	Wed 30 Sep	Eligibility, pre-window-0 features, sampling, buckets, manifest (U1–U8).
3	Thu 1 Oct	Model wrappers split into tune / fit; stubs and jitter deleted; strict failures; provenance; seed keys (U9–U13, U16, U17).
4	Fri 2 Oct	Task graph with scopes and seed check; sharding; merge; gates; analysis package incl. Friedman / Nemenyi and D26 labels, tested on synthetic data (U14, U15, U19, I1, I4).
5	Sat 3 Oct	Pilot on one GPU; I2, I3; gates; cost projection; go / no-go by the supervisor.
6	Sun 4 Oct	R0; R1 (prepare); start R2 (tuning) on four machines.
7	Mon 5 Oct	Finish R2; R2b freeze; start R3 (evaluation), cohort shards first.
8–9	Tue 6 – Wed 7 Oct	R3; validate shards as they finish.
10	Thu 8 Oct	R4 merge and gates; fix and rerun any failed task; buffer.
11	Fri 9 Oct	R5 analysis; tables and figures.
12–14	Sat 10 – Mon 12 Oct	Rewrite methods and results; supervisor review.

The critical path is days 1–5. A one-day slip in implementation removes one day of writing. The pilot’s go / no-go falls on a Saturday; if the supervisor is not available that day, R0 and R1 may start, since they do not depend on the decision, but tuning waits for it. A no-go costs one to two days and is escalated to the supervisor the same day.

11 Risks

Risk	Likelihood	Mitigation
Auto-class APIs differ in the pinned versions	medium	I5 on day 1. Fallback: an Optuna loop around the plain classes with the same spaces and objective, recorded in the change log.
Tuning overruns	medium	Measured in the pilot; reduce trials to 15 for the slowest family before R2 (D12).
Evaluation overruns	medium	Cohort shards first, seed check last; re-balance machines.
Machine preemption	medium	Per-task checkpoints; idempotent tasks; continuous sync to the bucket.
GPU non-determinism	high (small)	Fixed seeds, deterministic flags, forecast hashes; quantified by A15.
Degenerate buckets beyond nonlinearity	low	G12 flags them; handled as in D7.
Residual leakage	low	U1, U5, U12, G7.
Statistical failures on short series	low	Eligibility should remove them; U17 records any that remain.
Supervisor unavailable at a gate	low	R0–R1 may proceed; tuning and evaluation wait for sign-off.
Results contradict earlier drafts	high	Expected; the analysis plan is frozen before the run, so the conclusions follow the data.

12 Consequences for the paper

- **Methods to rewrite:** frozen data (D2), eligibility (D3), pre-window-0 features (D4, D5), per-feature sampling with M3 close to a census (D6), buckets and the support guard (D7), real tuning with its spaces and budget (D12–D14, D25), the two training scopes as a temporal hold-out on the same series (D15), strict execution and failure accounting (D16).
- **Sample sizes change:** 2,400 series per feature (1,200 per frequency, about 400 per bucket), about 8,500 distinct series overall.
- **All results sections are regenerated** from the frozen bundle. No number from any earlier version is carried over.
- **Language of effects:** “gain”, “loss”, “negligible” and “no evidence” are used exactly as defined in Section 5.3.
- **New figures:** critical-difference diagrams for both scopes (H7).
- **Cross-feature statements** (“feature A moderates more than feature B”) compare different samples and are worded as such; each feature’s gradient is reported on its own sample.
- **Limitations:** one main seed (quantified by A15); STL only; four features computed from STL components; M3 Quarterly close to a census; the lowered quarterly feature guards (D5, with A7); no feature-based selector (D28).
- **Reproducibility statement:** code, manifest and frozen results released (D29).
- **Unchanged:** introduction, related work, references, problem formulation, and the documented failure mode of tercile bucketing (now detected by G12).

A Defects found in the review

ID	Defect	Evidence
C1	Global baselines and statistical STL results are stub forecasts from a non-strict run; forecasts jittered per task.	<code>baselines.py:30–38;</code> <code>train_family.py:53–70;</code> <code>run</code> summary “NOT BENCHMARK-VALID”.

ID	Defect	Evidence
C2	No tuning; fixed configurations; placeholder Optuna files.	<code>mlforecast_auto.py:28-43</code> ; <code>neural_auto.py:34</code> ; <code>tune.py</code> ; empty <code>optuna/</code> .
C3	Features computed on $y_{1:n-h}$, seeing test windows 0–1.	<code>compute_all.py:66</code> .
C4	Seasonal lag and input size set by the shortest series in the pool.	<code>mlforecast_auto.py:45-50</code> ; <code>neural_auto.py:27-29</code> ; 33% of quarterly cohort fits had the lag.
M1	Trained global and tercile results on different series; no global STL-AC.	1,681 shared series; run configuration of 25 May.
M2	Candidate pool equal to the quota, so no stratification took place.	23 May configuration: pool cap 750.
M3	“Held-out series” wording; evaluation is a temporal hold-out.	<code>train_family.py:192-206</code> .
M4	Training histories as short as 6 quarterly and 12 monthly points.	<code>validation.py:6-16</code> .
M5	Thin neural training (500 steps, no explicit scaling).	<code>neural_auto.py:32-48</code> .
F1	Feature fallback values (0.0 / 1.0) assigned to short series.	1.7–2.9% of the 23 May sample.
F2	STL silently replaced by a constant trend or a moving average.	<code>mstl_features.py:28-51</code> .
F3	Pipeline source not under version control.	0 tracked files in <code>forecasting_</code> - <code>pipeline/src</code> .
F4	The data loader caught every exception and substituted synthetic series (for M4 first a local copy of the training file only, without the test period), without recording it; the smoke budget forced synthetic data. Latent: a scan of every stored run found synthetic series only in the six smoke runs (160 each); all pilot and large runs and all series behind the paper’s results (10,659 global-arm and 3,000 bucket-arm series) are real M3/M4 series. Found while implementing D2 (v1.3).	<code>load_m_datasets.py:50-85</code> and <code>run_experiment.py:66-67</code> at the baseline commit; scan of 14 feature tables and of <code>paper_</code> - <code>workspace/final_results</code> .
m1	STL-AC provenance taken from the trend model only.	<code>train_family.py:264-271</code> .
m2	Statistical known-limit fallback even in strict mode.	<code>statistical.py:64-83</code> .
m3	Rebuild merges runs without provenance checks.	<code>rebuild_paper_data.py:31</code> .
m4	Analysis scripts not versioned.	—
m5	No logs kept for the 28–30 May runs.	—
m6	M4 time stamps (integer positions) were converted to nanosecond date-times, so a task’s inferred frequency depended on which series came first. No effect on forecasts found (global training frames are rebuilt on a regular calendar and no model uses calendar inputs), but fragile. Found while implementing D2 (v1.3).	<code>load_m_datasets.py:88</code> ; <code>train_</code> - <code>family.py:235</code> ; <code>global_adapter_</code> - <code>data.py:26</code> .
m7	M3 values of the earlier runs were rounded to single precision by the TSF reader of <code>datasetsforecast</code> (29,051 monthly and 14,983 quarterly values; relative error $\leq 5.7 \times 10^{-8}$, i.e. the eighth significant digit). Found while implementing D2 (v1.3). Corrected in v1.4: v1.3 also listed the value –1200 of N2786 at $t = 84$ as a sign error in the Monash file; it is the value of the original M3C.xls.	Every value compared with M3C.xls; <code>np.float32</code> in <code>datasetsforecast/utils.py</code> .

B Decision record: alternatives considered

The questions put to D. Costa on 28 September 2026 (rows marked v1.3 or v1.4: 29 September), the option chosen, and the alternatives rejected. A rejected alternative may only be adopted later

through the change log.

Dec.	Question	Chosen	Rejected alternatives (and why they were on the table)
D1	Reuse old results?	Full rerun	Reuse the 25 May strict run and add missing arms (different samples, C3/C4 remain).
D2	Datasets	M3+M4 monthly and quarterly	Add M4 yearly as a negative control; add weekly/daily (needs MSTL).
D2	Data source	Frozen copy with check-sums	Download at run time (a changed mirror would change the data).
D2	M3 source (v1.4)	Original <code>M3C.xls</code> (IIF); Monash and Mcomp as cross-checks	Monash as the source (identical values, but a redistribution); Mcomp 2.7 (differs from the original in one value without documentation; chosen in v1.3 on the mistaken premise that Monash had an error); Monash through <code>datasetsforecast</code> , as in the earlier runs (single-precision rounding).
D2	M3 ids (v1.3)	Official M3 ids (N1402, ...)	Positional ids of the earlier runs (M1, ...), which do not match the literature.
D5	Short quarterly series	A: guards lowered to 20	B: two quarterly windows; C: original guards (M3 Quarterly ≈ 116); D: 400 per source.
D4	When features are computed	Once, before window 0	Recomputed every window (bucket membership would change).
D6	Sample pool	All eligible series	Random 5,000 per M4 source (faster).
D6	Sampling design	Per feature, 600 per source	One common 3,000-series sample (pairable across features).
D7	Nonlinearity feature	Keep definition, flag it	Make it continuous; break ties at random.
D7	Degeneracy threshold	Any bucket < 20%	< 10%; only an empty bucket.
D25	ML inputs	Lags $1 \dots m$	Lags + calendar dummies; lags + rolling statistics.
D10	Model set	Keep the 12	Add DLinear; add DLinear + a zero-shot foundation model.
D11	ARIMA search	Approximate	Exact search (slower; no regression reference).
D12	Tuning approach	Real light tuning (Auto classes)	Fixed configurations; tune once per family on Direct only.
D12	Tuning objective	MASE	RelNaive on validation (custom loss); MAE (not scale-free).
D12	Trials	20; 15 if pilot overruns	20 fixed; 30 fixed.
D13	Where to tune	Once, on pre-test data	Separately per feature sample.
D13	Validation data	One block of h	Two blocks monthly, one quarterly.
D13	STL-AC tuning	Separate study per component	One shared configuration; reuse the Direct configuration.
D15	Bucket-model tuning	Reuse the global configuration	Re-tune per bucket (confounds pool size with tuning).
D15	Experiment 2 scope	Fully paired	Reduced scope; move to the follow-up paper.
D15	Training pool	M3 + M4 pooled	Separate models per source.
D17	Seeds	One seed + seed check	One seed only; three seeds for neural and transformer.
D8	Refitting	Every window	Fit once, roll forward.
D16	Persistent failures	Fix and rerun the task	Exclude the configuration and report it.
D20	Machines	Four	Two; six or more.
D20	Sync target	Cloud bucket	University server; Google Drive.
D22	Library versions	Pin the May versions	Upgrade to the latest.

Dec.	Question	Chosen	Rejected alternatives (and why they were on the table)
D22	Repository	New folder pipeline/ in the existing project repository, branch rerun-v2 (amended in v1.2)	A new dedicated repository (the v1.1 choice, replaced on instruction of D. Costa).
D29	Release	Code, manifest and results	Code only; nothing until the follow-up.
D21	Primary metric	RelNaive ⁽¹⁰⁾	MASE; both co-primary.
D26	Practical threshold	0.01	0.02; no threshold.
D23	Inference	Wilcoxon + Holm + MCM + Friedman/Nemenyi	Wilcoxon only; Bayesian signed-rank with ROPE.
D23	CD-diagram scope	Cohort and tercile separately	Cohort only; families instead of configurations.
D27	Family pooling	Equal weights	Weight by model count; never pool.
D27	Descriptive level	Instances; tests per series	Series level everywhere.
D28	Selector	None in this paper	A simple exploratory rule.
D24	Freeze	Tagged commit	Public pre-registration on OSF.
D30	Sign-off	Supervisor	Supervisor + partner; D. Costa alone.

C Change log

Version	Date	Change
1.0	28 Sep 2026	First draft for approval.
1.1	28 Sep 2026	All open decisions taken in eight rounds of questions: D5 confirmed (option A); new decisions D25–D30 (ML inputs, practical threshold, aggregation, selector, release, sign-off); D2, D8, D11–D13, D15–D17, D20–D24 made explicit; Friedman + Nemenyi rankings (H7) and seed check (A15) added; gates G13 and tests U18–U19 added; timeline moved to start on the day of sign-off; Appendix B added.
1.2	28 Sep 2026	D22 amended on instruction of D. Costa: the pipeline lives in the new folder pipeline/ of the existing project repository (github.com/DiogoCosta18/TimeSeriesForecasting), branch rerun-v2 , instead of a new dedicated repository. Consequence: sharing the code with the partner university means sharing the repository, paper workspace included. Sections 1.2, 4.6, 6.1, the timeline and Appendix B updated accordingly.
1.3	29 Sep 2026	D2 amended on instruction of D. Costa after the M3 comparison: M3 from Mcomp 2.7 (CRAN) with official ids, the Monash copy kept as a bit-for-bit cross-check; rdata 0.11.2 and xarray 2026.7.0 added to the lock (no other pin changed); Appendix B extended. Recorded while implementing D2: defects F4 (silent synthetic-data fallback in the loader; shown to be latent), m6 (M4 time stamps) and m7 (M3 values of the earlier runs) added to Appendix A and to the traceability table; Section 3 describes the frozen copy as implemented (sources, validation, file layout, manifest, order-only calendar); U18 extended; the rule for inherited tests and the four failing baseline tests stated in Section 7; module table updated.

Version	Date	Change
1.4	29 Sep 2026	Correction of 1.3. The statement that the Monash file carries a sign error was wrong: the original competition file <code>M3C.xls</code> (IIF), obtained the same day, has -1200 at $N2786$, $t = 84$, and Monash reproduces it exactly; Mcomp 2.7 is the copy that differs ($+1200$, undocumented). D2 decided again by D. Costa: M3 from <code>M3C.xls</code> (pinned SHA-256), with Monash (identical) and Mcomp (one documented difference) as cross-checks that stop the freeze on any other difference. Section 3, the decision register, defect m7, the environment paragraph, the module table and Appendix B corrected accordingly.
1.5	1 Oct 2026	Sign-off: v1.4 approved by the supervisor (reported by D. Costa on 1 October 2026); the approved text is the commit tagged <code>protocol-v1.4-approved</code> . From this version on, the body keeps the approved text and amendments are listed here only. Amendments: (a) D20: the cloud bucket is a private Backblaze B2 bucket, accessed with <code>rclone</code> (decision of D. Costa, 1 October); credentials are created by D. Costa and never stored in the repository. (b) Section 6.3: the frozen data copy is created and checked with <code>python -m src.data.freeze_data freeze --out DIR --code-commit SHA --m3c M3C.xls</code> and <code>... verify --data-dir DIR</code> (already done: copy <code>m3m4-v1</code>), then uploaded to the bucket with <code>rclone</code> ; the <code>freeze-data</code> line of the listing becomes a <code>src.cli</code> sub-command with the same arguments when the stages are split (day 4). (c) Section 4.6: the lock also pins <code>sympy 1.13.1</code> , the version <code>torch 2.5.1</code> requires (the 21 May freeze had 1.14.0, which came with a later torch); <code>pipeline/environment/README.md</code> gives the derivation. Appendix C is typeset as a table that can break across pages.

Version	Date	Change
1.6	3 Oct 2026	Implementation of days 1–2; no decision changed. (a) D20: the frozen copy is in the private Backblaze B2 bucket (path <code>data/frozen_m3m4_v1/</code>, 235 MB), verified by <code>rlone</code> (SHA-1, then byte for byte) and by <code>freeze_data verify</code> on a fresh download. (b) D4, F1, F2 as implemented: besides the fallbacks named in Section 6.2, the feature code substituted values on short or constant input, computed other statistics when a library call failed (moments, squared autocorrelation, and a full scan for ruptures that kept the flag “ok”), interpolated missing values, set seasonal strength to 0 when undefined, and turned series exceeding a 120 s timeout into fallbacks, so that eligibility depended on machine speed. All removed: a feature is “ok”, “undefined:reason” or “error:type”, with no value otherwise, and there is no time limit. On real data every “ok” value equals the earlier code bit for bit except the D5 cases. Consequence of F2: a series with a constant rolling window (36 monthly, 16 quarterly points) has no evolving-seasonality value and is ineligible. A failed parquet write no longer produces a CSV file under a parquet name. (c) Seeds: “seed = $s_0 + k$” ($s_0 = 20260521$) is the numpy seed tuple ($s_0 + k, m, \text{source}$), one stream per feature, frequency and source; the tuning set uses ($s_0, 1000, m, \text{source}$). (d) Buckets: the cut points are the values at positions $\text{round}(n/3)$ and $\text{round}(2n/3)$ (ties by series id), tied values share a bucket, and an empty bucket counts as below 20% (Appendix B: “any bucket < 20%”). A pool below 600 stops the stage; nothing is filled from elsewhere. (e) Verification run of stage R1 at commit <code>cb5c047</code> (bundle <code>de54ccf5...</code>): eligible M3 Monthly 1,039, M4 Monthly 35,010, M3 Quarterly 634, M4 Quarterly 22,591, in total 59,274 (Table 4 estimated 1,040, 35,000, 634, 22,500). Ineligible after the length rule: 110 M4 Monthly series (one constant throughout, 109 through evolving seasonality only) and 21 M4 Quarterly. 8,492 distinct sampled series; nonlinearity degenerate in both frequencies (Medium empty; Low 1,007 monthly and 976 quarterly), as Section 3.5 expects; every other feature 400/400/400; Kolmogorov–Smirnov $p \geq 0.9999$ for every sample (G11). (f) Until the stage CLI of Section 6.3 exists, the stage runs as <code>python -m src.stages.prepare</code>.

Version	Date	Change
1.7	3 Oct 2026	Implementation of day 3; no decision changed. (a) The model layer is the new package <code>src/forecast/</code> (registry, the search spaces of Table 6, result object, fits from frozen configurations, tuning, metrics, evaluation engine). The legacy modules of Section 6.2 (<code>src/models/*</code>, <code>train_family.py</code>, <code>baselines.py</code>, the component stub, the jitter, strict mode) are deleted when the runner switches to it on day 4. (b) Global models receive the positions t as time stamps (frequency 1). (c) Tuning: the neural training loss is MAE, the models’ default (the validation loss is MASE as specified); early stopping uses a patience of 5 validation checks every 50 steps, values the protocol does not fix; the frozen configuration holds the number of steps the best configuration actually trained, read from its refit on the same validation block; the TPE sampler is seeded per study from s_0 and every fit uses s_0 (D17); for the STL targets, STL is fitted once on each tuning series up to its first cutoff; every trial is archived in SQLite with its parameters, validation MASE, timestamps and duration. (d) Determinism (U10): forecasts are reproducible bit for bit on CPU. RandomForest runs on one thread (with several, the tree predictions are summed in completion order and the last bits vary) and XGBoost on a fixed four. On a GPU there is no deterministic CUDA median (robust scaler), so the deterministic flag is “warn” there and the seed check (A15) quantifies the variation; the device is recorded with every forecast. (e) Statistical models are fitted with fit and predict, which give the earlier runs’ <code>forecast</code> results bit for bit, so regression test I2 is unaffected. (f) POCID is computed as the paper’s equation. Defect m8: the code that produced the paper’s POCID values (<code>rebuild_paper_data.py</code>) used $h - 1$ steps without the anchor on the last training value and counted a zero change on both sides as agreement, so the published values do not follow the published equation. (g) Smoke run on real data (quarterly, one feature sample of 1,200 series, laptop RTX 3050 Ti): no failures; Ridge 1.5–9 s per task of three windows, NHITS on GPU 31 s, statistical fits 0.03–0.18 s per series and window.

Version	Date	Change
1.8	5 Oct 2026	Implementation of day 4; no decision changed. (a) Layout: the stages are the new package <code>src/stages/</code> (prepare, tune, freeze, evaluate, merge, task graph, run records), the gates are <code>src/validation/gates.py</code>, the analysis is <code>src/analysis/</code>, and <code>src/cli.py</code> is the interface of Section 6.3 (prepare runs only through it, with the commit read from a clean git tree). The legacy modules of Section 6.2 were replaced rather than refactored and then deleted with their tests, not switched off. What the existing tests of Section 7.1 guarded is tested on the new code: checkpointing and scheduling by the stage resume and failure tests and I4; the adapters by U10, U11 and U17; the Optuna status by U11 and U12; strict provenance by U9, which now also asserts that the legacy packages are absent and that no module holds a fallback, substitute or jitter path; timing by a merge check; benchmark correctness by the task-graph, engine and model tests; sampling and scaling by U7 and U8; STL by U5 on the engine’s own targets. (b) The seed check uses $s_0 + 1$ and $s_0 + 2$ (20260522, 20260523). (c) Grid of the verification bundle: 324 cohort global tasks, 918 tercile tasks (nonlinearity degenerate in both frequencies), 84 seed-check tasks; the 108 statistical cohort views are served by 315 chunks of at most 250 series (4,491 monthly and 4,001 quarterly series, 3 models, 3 strategies). (d) D16 and G4: evaluation runs at the commit, environment lock and data manifest of the frozen configurations (the R0 commit). Another commit is accepted only to rerun listed failures; it must descend from that commit, nothing else may differ, and each such rerun is entered in the shard’s register <code>d16_reruns.json</code>. G4 accepts a second commit only for registered reruns. The 90 studies must share one provenance, so a fix during tuning means tuning again. First attempts that failed, listed failures and resolved failures are all kept for A8. (e) Every stage checks the configuration, the data manifest and every prepare file against the prepare bundle and refuses a mismatch. The merge also refuses outputs that changed or are not in the grid, rows that disagree with their task or whose forecasts do not match their hashes, trained rows without a positive training time, and studies that no longer match the frozen configurations. An incomplete grid is merged and caught by G5. A bundle whose gates passed is never rebuilt, and the analysis refuses a bundle without a passing, current gate report. (f) Gates as implemented: G1 also checks the number of component provenances per row; G6 pairs trained and failed rows; G7 requires the feature and tuning ends to be at most the first cutoff and every study to carry its check; the capped share of G9 is the share of trained rows with RelNaive above 10; G10 is the median, per family, of the STL-AC/STL-SN training-time ratio over paired task windows; the Kolmogorov–Smirnov tests of G11 are computed by the gates; G12 recomputes the bucket sizes; G13 compares the bundle’s data record with the manifest.

Version	Date	Change
		(g) Analysis conventions the protocol leaves open: Wilcoxon drops zero differences, and a contrast without a non-zero difference, like a Spearman correlation with a constant variable, has $p = 1$ (no evidence); every bootstrap stream is seeded from s_0 and the contrast’s name; Jonckheere–Terpstra uses the tie-corrected normal approximation, two-sided; in H4 the trend test leaves out degenerate frequencies (D7), so nonlinearity, degenerate in both, is listed as excluded, while Spearman uses all series; the interval of the High–Low contrast resamples series within each tercile; H3 contrasts are RelNaive⁽¹⁰⁾(first)–RelNaive⁽¹⁰⁾(second), positive favouring the second strategy; in H6 the families are weighted equally within each model type, with the linear models of Table 5 (Ridge, LinearRegression, NLinear); H7 uses the instances where every configuration has a value, rank 1 being the lowest per-series mean RelNaive⁽¹⁰⁾; in A5 unclipped RelNaive is aggregated per series by the median; the trimmed means of A6 drop 5% at each tail; A7 applies Holm over its own tests; A13 applies Holm within each table; in A15 the deterministic models (statistical, Ridge, LinearRegression) keep the main seed’s forecasts, and labels are compared on the cohort-scope evolving-seasonality sample. (h) Pilot: the configuration key features restricts the features that are sampled and evaluated; eligibility still requires all six, and each feature is drawn with its full-run seed. <code>configs/pilot_v2.yaml</code> takes 25 strata of 4 series per source, because 100 cannot fill 30 equal strata; a test checks that it differs from the full-run configuration only in the settings of Section 8.1. <code>prepare --jobs auto</code> counts the CPUs of the container (cgroup quota), not those of the host. (i) D20: <code>scripts/sync_run.sh</code> uploads the run directory every 5 minutes with checksums, using rclone copy, which never deletes in the bucket, and ends with a check that every local file is in the bucket; <code>scripts/fetch_run.sh</code> downloads a run or parts of it and checks them; <code>scripts/run_stage.sh</code> keeps each stage’s log inside the run, so logs are synced too. (j) The seasonal naive refuses fewer than m training values (it returned zeros or the last value); this cannot happen for an eligible series. (k) Tests: U14 and U15 (merge), U19 (Holm against statsmodels; Wilcoxon and Friedman against scipy; Jonckheere–Terpstra mean and variance against exact enumeration; Nemenyi’s q against Demšar’s table), I4 (a shard killed mid-run and resumed gives the uninterrupted outputs apart from timings, and nothing finished is recomputed) and I1 (the whole chain through the command line on synthetic data, with real fits of all twelve models on CPU and one trial per study, passes with every gate passing in 30 minutes on a laptop; only for this test the neural models use 20 steps and the standard scaler). Suite: 299 unit and stage tests and the 2 integration tests pass (commit 0ba418f).

Version	Date	Change
1.9	5 Oct 2026	Pilot (day 5). Three amendments decided by D. Costa on 5 October 2026, (c), (d) and (e); otherwise no decision changed. (a) Machine: one vast.ai instance with an RTX 4090 (24 GB) and an AMD EPYC 7B13, limited by its container to 61.2 CPUs (a cgroup v1 quota, which <code>prepare --jobs auto</code> now reads), image <code>pytorch/pytorch:2.5.1-cuda12.1-cudnn9-runtime</code> (Ubuntu 22.04), used for every machine of the full run (D22). <code>scripts/setup_machine.sh</code> built the environment from the lock (I5 passed) and fetched and verified the frozen copy; <code>scripts/run_all.sh</code> ran every stage. (b) First pilot, commit <code>04e5486</code>, <code>configs/pilot_v2.yaml</code>: prepare 4 minutes, the 90 studies 40 minutes, 414 evaluation tasks with 150,525 trained rows, no failed row, no retried task; gates G1–G9 and G11–G13 passed, G12 flagging nonlinearity in both frequencies as expected; G10 failed, see (d). (c) Platform. With identical code and inputs, AutoARIMA gives different forecasts on different processors: on 800 sampled May rows an Intel i7-11800H and the AMD EPYC 7B13 agreed bit for bit on 245, because the stepwise model search turns last-bit differences (processor-specific OpenBLAS kernels, numba compiled for the host) into different models; ETS differs by about 10^{-8}. Amendment: <code>src/__init__.py</code> pins the OpenBLAS kernel family (Haswell), numba’s target (haswell) and single-threaded BLAS before numpy loads (agreement between the two machines then 791 of 800; speed unchanged); every completion record and the prepare bundle record the platform (processor, libc, the kernel and threads each loaded OpenBLAS reports, numba’s target) and whether the pins were in effect; G4 also requires the pins in effect for every task and one processor model per statistical shard. (d) G10. Each STL target is tuned separately (D12–D14) and trains its own early-stopped number of steps, so the STL-AC/STL-SN time ratio follows the ratio of trained steps (LSTM 3.64 against 3.60; NLinear 7.90 against 7.60) or of the tuned ML configurations, not three models against one. Amendment: G10 checks directly that every trained row holds the component forecasts its strategy defines (Direct: raw; STL-SN: non-seasonal; STL-AC: trend, seasonal, residual) and that every global component carries the configuration and trained steps of its own target’s frozen study (now recorded per component); the time ratio, divided by the step ratio for neural and transformer models, is reported in A9.

Version	Date	Change
		(e) I2. Because of (c), the 25 May values cannot be reproduced for AutoARIMA on another processor. Amendment: I2 compares the code of the 25 May run (baseline <code>f1eb88b</code>) with the new code on every eligible May row of AutoARIMA and AutoETS Direct, with the inputs as that run read them (datasetsforecast, M3 with its float32 rounding, m7), the windows checked against D8, both under the same pins; it passes if every forecast is bit-identical. Result: 38,646 of 38,646 rows identical (49,153 May rows, 10,507 no longer eligible); agreement with the stored May values, within 10^{-6}: AutoETS 12,522 of 12,522, AutoARIMA 6,455 of 12,522 monthly and 10,626 of 13,602 quarterly. (f) Re-balancing (Section 8.3): one process needs about 2.4s per series and window for SARIMA under STL-AC (0.3s Direct; the residual component dominates), so the statistical shards would take most of a day on the full run; <code>evaluate --workers</code> computes the tasks of a CPU shard in several processes, the main process alone writing outputs and records (outputs byte-identical to one process, tested); completion records keep each task's compute time. (g) D20: the five-minute uploads exhausted the bucket's daily transaction cap at 16:50 UTC; D. Costa raised the cap, and every upload and check now lists the bucket once (<code>--fast-list</code>). The first pilot was copied to the laptop and verified file by file (1,071 files), then uploaded and checked. (h) GPU memory over the whole discrete search grid of each neural and transformer model, at the largest input and batch sizes and the full run's pools: at most 2.2 GB (TFT monthly) of 24 GB, no out-of-memory error in 126 fits. (i) Second pilot at the final commit <code>4da81f2</code>: with 18 statistical and 4 ML worker processes, prepare was byte-identical to the first pilot, all 90 frozen configurations were identical, and every one of the 150,525 forecasts equalled the first pilot's bit for bit, in every family, GPU-trained models included. Every gate passed: G4 with the pins in effect on every task and one processor model per shard, G10 with no row out of place. The analysis produced every output of Table 8 (I3). Wall times: prepare 4 minutes; tuning 38 minutes; evaluation 3.3 hours along the GPU shards (0.6, 0.5, 1.0 and 1.2 hours), 35 and 14 minutes for the statistical shards, 18 and 2 minutes for ML; no failed row, no retried task. Both pilots are in the bucket and on a second machine, checked file by file. Pilot results are never reported. (j) Projection of the full run on machines A–D (Section 9.3), from the measured times with the assumptions stated in <code>scripts/pilot_report.py</code>: 13.4 hours of wall-clock time (criterion: at most 72), set by machine D (ML tuning 7.3 hours and ML evaluation 6.1 hours with 4 workers; statistical 1.3 hours with 20 workers per shard); machines A, B and C 3.3, 9.5 and 5.8 hours. Neural and transformer times are scaled by 1.26, the largest growth measured on the pilot machine when the pool goes from 200 to 1,200 series (0.74–1.26; they train a fixed number of steps); without that measurement the bound would be 45 hours. The exit criteria of Section 8.3 are met; the decision is the supervisor's (D30). Tests: 306 unit and stage tests and the integration tests I1 and I4 pass at commit <code>4da81f2</code>.

Version	Date	Change
2.0	6 Oct 2026	Full run (day 6). The supervisor approved the go and the amendments of v1.9 on 5 October 2026. No decision changed; (c) is a defect fixed before any result. (a) Re-balancing (Section 8.3): the projection of v1.9 added each machine’s stage times, but no evaluation can start before the 90 studies are frozen, so the critical path was ML tuning (6.7 hours, almost all of it five RandomForest studies run one after another) followed by the longest evaluation shard: about 15 hours, not 13.4. <code>tune --workers</code> runs the studies of an ML shard in several processes; each study keeps its own TPE and fit seeds and runs its trials in order, and the main process alone archives the trials and writes the entries and records (commit 29abfa6; on a real ML shard the entries are byte-identical and the archived trials equal to those of one process, tested); workers are refused on GPU shards. (b) Machines (D22): four vast.ai instances with an RTX 4090 and the pilot’s image. D, the lead, on the pilot’s host (AMD EPYC 7B13): prepare, ML tuning (20 processes per shard), freeze, ML evaluation (8 processes) beside the statistical shards (20 processes), then merge, gates and analysis; A (EPYC 7763) the neural shards; B (EPYC 9V74) transformer quarterly; C (EPYC 7Y83) transformer monthly. A first machine C refused every SSH connection and was replaced before it ran anything. Each machine was set up by <code>scripts/setup_machine.sh</code> (I5: 38 tests passed, lock hash 84c729b1; frozen copy verified). <code>scripts/run_machine.sh</code> (commit 5a5a9fa) runs one machine’s part without an operator: the machines coordinate only through the bucket (D20); each step publishes its outputs, checks them file by file against the bucket and writes a marker; a machine that needs another’s outputs waits for the marker, then fetches and checks them; a failed step writes a failure marker, the waiting machines stop and nothing is merged; a machine stops itself, with a key limited to that instance, only after its last upload was checked.

Version	Date	Change
		(c) Defect found at the start of the run and fixed before any result: the first start (<code>full_20261005</code>, commit <code>191940c</code>) stopped in ML tuning. With 20 trials, <code>mlforecast</code> reports each trial's only validation window to Optuna, whose default median pruner (active once five trials exist) ended some trials as pruned, without a value; the archive refuses such trials (Section 5.1), so all ten LinearRegression and Ridge studies of ML monthly failed twice within three minutes (e.g. <code>Ridge monthly raw</code>: trials 8, 9, 14, 15, 17 and 19). The pilots ran three trials and never reached the pruner. With one validation block pruning saves no computation; it only drops trials' values and changes what TPE sees. Fix (commit <code>cafc5cc</code>): the ML studies use no pruner, so every trial is evaluated and archived as Section 4.3 states; <code>neuralforecast</code> reports no intermediate values and cannot prune. A regression test with the full run's 20 trials fails without the fix (trials 7, 8 and 16 pruned). Every stage was stopped and the run restarted from prepare at <code>cafc5cc</code> as <code>full12_20261005</code>, so all studies and tasks share one commit (G4). The aborted start stays in the bucket with a record of what happened; its last upload predates the failures, so its own failure log was lost with machine D. (d) Platform note on prepare: the bundles of both starts (same host) are byte-identical. Against the verification bundle of day 2 (commit <code>cb5c047</code>, Intel laptop, before the pins of v1.9), eligibility, cut-offs and the tuning set are identical; features differ in their last bits (spectral entropy by up to 7×10^{-4} in 121 of 59,404 series), which moves 6 of the 14,400 sampled series across a stratum boundary (2 for the ARCH statistic, 4 for non-normality; the other four samples are identical). As for AutoARIMA (v1.9 (c)), the bundle is computed once on one machine, hashed and used by every stage, and its platform is recorded.

Version	Date	Change
		(e) The full run was restarted on 5 October at 23:28 UTC as <code>full12_20261005</code> (commit <code>cafc5cc</code>, <code>configs/rerun_v2.yaml</code>). Prepare 4 minutes. The 90 studies: none failed or was retried; ML monthly 3.5 hours with 20 processes (the critical path), ML quarterly 0.3, neural 0.9 and 0.8, transformer 1.3 (monthly) and 1.0 hours (quarterly); frozen at 03:21 UTC. Evaluation, 1,641 of 1,641 tasks (324 cohort, 918 tercile, 84 seed check, 315 statistical chunks): statistical 1.8 and 0.6 hours (20 processes), ML 6.2 and 0.4 hours (8 processes), neural 3.6 and 3.0 hours, transformer 5.4 and 3.6 hours; 2,864,511 trained rows, no failed row, no task retried, no listed failure, no D16 rerun. Merge 1 minute (result hash <code>7cab5f83</code>); every gate G1–G13 passed (G4: one commit, one environment, the pins on every task and study, one processor model per shard; G11: 24 Kolmogorov–Smirnov tests, smallest p 0.9999; G9: 0.10% of metric values capped); the analysis produced all 61 outputs of Table 8 (I3) in 7 minutes. Wall-clock time 10.7 hours from the restart (criterion: at most 72). (f) The run (3,612 files, 2.7 GB) is in the bucket and on the laptop, checked file by file (no difference); the four machines were then destroyed. Cost of day 6, aborted start included: about US\$22. (g) After the run: <code>publish</code> in <code>scripts/run_machine.sh</code> mixed <code>include</code> and <code>exclude</code> rules, and <code>include</code> applies every <code>include</code> before any <code>exclude</code>, so its exclusion of temporaries never applied inside a published path. None existed when a step published (stages write atomically and had ended), and none of the run’s 3,612 files is a temporary. Ordered filter rules since commit <code>dad1056</code>, with a test. Tests: 314 unit and stage tests pass at commit <code>dad1056</code>; the integration tests I1 and I4 last passed at <code>4da81f2</code>, and the full run exercised every stage.
2.1	6 Oct 2026	After the run (days 7 and 8). No decision changed and no result of v2.0 changed. (a) Machine D was destroyed once the run was verified in the bucket and on the laptop; no rented machine remains. (b) Analysis figures (commit <code>8940a69</code>): the D26 label of each bar is a word placed beyond the end of its interval (it used to sit on the interval’s upper end and read as a second cap), and the tercile figure has one column per feature and one row per strategy, with a tercile that holds no series (nonlinearity, D7) marked as empty instead of drawn as zero. Figures only. (c) The analysis reads only the 24 merged-row columns it uses (commit <code>269f236</code>); the merged files are still checked whole against the merge record, and the gates still read every column. Reproduction: on a laptop with 8 GB of memory (7.4 GB usable by Linux), loading every column had been stopped by the out-of-memory killer. At <code>269f236</code>, from a clean clone, the analysis of <code>full12_20261005</code> then took 7 minutes and produced the 61 outputs; all 43 tables are byte-identical to those computed on machine D at <code>cafc5cc</code>, as are 14 of the 18 figures, the other four being those changed in (b) (the compute-cost chart shares the bar layout). The regenerated analysis is kept as <code>analysis_269f236</code> beside the original, in the bucket and on the laptop, checked file by file.

Version	Date	Change
		(d) Exploratory analyses, not pre-registered (commit 89bf7e3, <code>src/analysis/exploratory.py</code>). They were added after the results were known, to answer questions of the earlier paper that Section 5 does not cover; they test no hypothesis and change no label, and every output says so. They use the gated instances and the aggregation rules of Section 5 (cohort scope, main seed, per-series means). E1: for each series and model, the strategy with the lowest RelNaive and the gain of that oracle choice over Direct, an upper bound because choosing the best of three noisy estimates favours the oracle. E2: STL-SN when a series' evolving seasonality is below its frequency's first tercile cut, Direct otherwise; the rule was chosen after seeing H4 on the same data, so its gain is in-sample and only illustrative. E3: mean and median RelNaive and mean POCID per model and strategy. Outputs: <code>exploratory_2b31298</code>, in the bucket and on the laptop, checked; the paper reports them as exploratory. The first output, <code>exploratory_269f236</code>, is kept: its tables and figure are byte-identical, but its status line named Section 7 instead of Section 5, see (f). (e) Observations for the paper, not changes: the penalty range of Ridge in Table 6 (10^{-3} to 10^2) is negligible at the scale of the lag features, so Ridge and LinearRegression give RelNaive values within 10^{-5} of each other on 90% of 259,200 paired forecasts; the run followed the approved space, so nothing is rerun and the paper states it. Every sMAPE effect is labelled negligible because the D26 threshold is defined on the RelNaive scale. (f) References corrected (commit 2b31298): the analysis code called the outputs table “Table 6” (it is Table 8; Table 6 holds the search spaces), the gates module called the gates “Section 7.4” (they are Section 7.3), and the exploratory module named Section 7 as the confirmatory plan; docstrings, one error message and the exploratory status line only. Tests: 319 unit and stage tests pass at commit 269f236; the analysis, exploratory and gate tests pass again at 2b31298.

Version	Date	Change
2.2	7 Oct 2026	Sensitivity analysis S1, the STL configuration: decided by D. Costa on 7 October 2026, after the results of v2.0 were known, and specified here before any of its forecasts exists. No decision of the run changes and S1 adds no confirmatory hypothesis; the paper reports it as a sensitivity analysis. Question: do the STL conclusions (H1, H3) depend on the STL of D9, robust STL of the raw series with statsmodels' default seasonal smoother? Variants of STL-SN: (L) log STL, STL of $\log y$ (every series is positive), the forecast being exp of the non-seasonal forecast plus the seasonal continuation, for seasonality that grows with the level; (P) periodic STL, seasonal smoother of length $10n + 1$ and degree 0 on a training window of n points (R's <code>s.window = "periodic"</code>), a seasonal pattern fixed over the window. Both robust, fitted per window on training data only, a failure being a failure (D9, D16). Models: the one with the lowest median RelNaive under Direct in each family (A1): ETS, XGBoost, NHITS and PatchTST. Scope: STL-SN only; cohort scope, main seed, every feature sample, both frequencies and the three windows, with the series, cutoffs, buckets and Direct forecasts of <code>full12_20261005</code> (Direct is not recomputed). Tuning: one study per global model, frequency and variant on that variant's non-seasonal target, with the settings, seeds and search spaces of Section 4.3 and Table 6 (12 studies); ETS is not tuned. Analysis: for each model and variant, Δ_{STL} against the run's Direct forecasts, a Wilcoxon signed-rank test on per-series means, Holm over the 8 tests, the median with a bootstrap 95% interval and the D26 label; beside it, the same statistics for the run's own STL-SN of these models; medians by evolving-seasonality tercile, descriptive only. Checks before the analysis, as gates G1, G4, G5, G6 and G9 for S1: every task complete, every forecast paired with its Direct row, finite metrics, one commit, the processor pins in effect. Decision rule: the STL conclusion stands if no model has the label <i>gain</i> under either variant; otherwise the paper states for which models and variants the effect of STL depends on the configuration.

Version	Date	Change
2.3	8 Oct 2026	After S1 (day 9). No decision of the run changes and no result of v2.0 changes. (a) S1 ran as specified in v2.2 on one rented machine at commit <code>b6bd978</code>: the 12 tuning studies and the 142 evaluation tasks completed with no failure and no retry. The outputs were synced to the bucket, copied to the laptop and checked file by file (the run folder now holds 4,016 files, with no difference between bucket and laptop); the machine stopped itself and was then destroyed. S1 cost about US\$0.8. (b) The S1 analysis ran on the laptop from a clean clone at <code>2f05a33</code> (the S1 code is unchanged since <code>b6bd978</code>). Checks G1, G4, G5, G6 and G9 passed; the tables are in <code>sensitivity_s1/analysis</code>, uploaded and checked. (c) Result, median per-series Δ_{STL} against the run's Direct forecasts with its D26 label. Log STL: ETS -0.100, NHITS -0.099 and PatchTST -0.096, all losses; XGBoost -0.014, no evidence. Periodic STL: ETS -0.002, negligible; NHITS -0.010 and PatchTST -0.031, losses; XGBoost $+0.038$ (95% interval $+0.034$ to $+0.042$), a gain. With the default STL of D9 the same four models give -0.079, -0.086, -0.093 and -0.007. (d) Decision rule of v2.2: one model has the label <i>gain</i> (XGBoost under periodic STL), so the paper states that for XGBoost the effect of STL depends on the configuration. For ETS, NHITS and PatchTST no variant turns the loss into a gain, but periodic STL makes the losses of all four models smaller. The confirmatory results H1 to H7 are unchanged; S1 is reported as a sensitivity analysis. (e) Exploratory analyses E4 to E6 (commit <code>2f05a33</code>), not pre-registered and added on 8 October after the results were known: E4, an honest version of the oracle E1 that chooses each series' strategy on windows 0 and 1 and scores the choice on window 2; E5, median per-series Δ_{STL} in cells of one or two feature terciles; E6, the ML family without Ridge and without LinearRegression, because the two give almost the same forecasts (v2.1 (e)). Their tables are in <code>exploratory_2f05a33</code> with content hashes; the outputs of E1 to E3 there are byte-identical to those of <code>exploratory_2b31298</code>.